Audit système et déploiement

BUT informatique 3A, parcours DACS, mini-rapport

Binôme : Craincourt Noa / Taleb Camil
Octobre 2026

1 Audit système

Le but de cette première partie est de décrire toutes les commandes dont nous aurons besoin pour la suite du projet.

Notre méthode pour trouver les commandes

Nous ne sommes pas très familiers avec Linux, donc beaucoup de ces commandes nous étaient inconnues. Avant de nous lancer, nous nous sommes fixé une méthode : chercher un mot-clé avec `apropos`, puis lire la page `man` correspondante pour savoir comment obtenir l'information voulue. Pour la version du noyau, par exemple, `apropos kernel` nous a fait découvrir `proc_version`. `man proc_version` nous a ensuite appris qu'il suffisait de faire `cat /proc/version`. Cette méthode nous a donné le réflexe de chercher par nous-mêmes au lieu d'aller directement sur Google, qui reste notre solution de secours quand nous ne trouvons vraiment rien.

Nom de la machine

La commande est `hostname`. Elle retourne « debian-noa ». Pas besoin de sudo.

Distribution (nom et version)

`cat /etc/os-release` donne toutes les informations, dont celle que nous cherchons :

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.7
ID=debian
```

`lsb_release -d` fonctionne aussi et retourne « Description: Debian GNU/Linux 13 (trixie) ». Nous préférons `/etc/os-release` : comme l'explique le sujet, `lsb_release` dépend du paquet `lsb-release`, absent des installations minimales et de la plupart des images Docker. Pas besoin de sudo.

Version du noyau

Nous avons expliqué plus haut, avec l'exemple de notre méthode, comment nous avons trouvé `/proc/version` :

```
noa@debian-noa:~/PreSaeNoa$ cat /proc/version
Linux version 6.12.113+deb13-amd64 (debian-kernel@lists.debian.org) (x86_64-
linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU ld (GNU Binutils for Debian)
2.44) #1 SMP PREEMPT DYNAMIC Debian 6.12.113-1 (2026-10-15)
```

Nous avons aussi testé `uname -r`, où l'option `-r` donne la version du noyau en cours d'exécution :

```
noa@debian-noa:~/PreSaeNoa$ uname -r
6.12.113+deb13-amd64
```

Nous utiliserons `uname -r`, car sa sortie donne exactement ce dont nous avons besoin, alors que celle de `cat /proc/version` est brute. Pas besoin de sudo.

Uptime et charge moyenne

Pour l'uptime et la charge moyenne, la commande `uptime` porte bien son nom :

```
noa@debian-noa:~/PreSaeNoa$ uptime
17:52:29 up 1:03, 3 users, load average: 0,01, 0,04, 0,01
```

Elle donne le temps écoulé depuis le démarrage de la machine (ici 1 h 03), puis la charge moyenne du système sur la dernière minute, les 5 dernières et les 15 dernières. Notre système était plutôt tranquille.

Mémoire et swap

Pour la mémoire et le swap disponibles et utilisés, nous utilisons `free`. Nous ne la connaissions pas avant ce projet : nous l'avons découverte en testant les commandes citées dans les commentaires de la partie 1 du sujet. En lisant `man free`, nous avons aussi appris que les mêmes informations se trouvent dans `/proc/meminfo`. Voici ce que retourne la commande :

```
noa@debian-noa:~/PreSaeNoa$ free -h
               total        utilisé      libre       partagé  tamp/cache   disponible
Mem:           1,9Gi       1,0Gi       489Mi       584Ki       553Mi       905Mi
Échange:       1,1Gi       0B          1,1Gi
```

L'option `-h` choisit automatiquement l'unité la plus lisible pour un humain : B pour octets, Ki pour kibioctets, Mi pour mébioctets, Gi pour gibioctets.

Le total de « Mem » correspond à la mémoire allouée à notre système (nous sommes dans une VM, à laquelle nous avions donné 2 Go). Le total du swap correspond à l'espace de secours réservé sur le disque de la VM. « Utilisé » est la mémoire utilisée ou indisponible, et « libre » la mémoire inutilisée. « Partagé » est la mémoire utilisée surtout par tmpfs. « tamp/cache » est la mémoire utilisée par les tampons du noyau, le cache de pages et les slabs. Enfin, « disponible » est la mémoire disponible pour lancer de nouvelles applications sans utiliser le swap.

La différence entre libre et disponible : la mémoire disponible additionne la mémoire libre et la mémoire utilisée temporairement, puisque cette dernière peut être récupérée en cas de besoin. Pour le swap (« Échange »), nous avons cette fois cherché sur Google : il n'a tout simplement pas de cache, d'où l'absence de colonne « disponible ». Pas besoin de sudo.

Interfaces réseau (adresses MAC et IPv4/IPv6)

```
noa@debian-noa:~/PreSaeNoa$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 08:00:27:12:33:59 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute
enp0s3
       valid_lft 74276sec preferred_lft 63476sec
    inet6 fd17:625c:f037:2:a00:27ff:fe12:3359/64 scope global dynamic
mngtmpaddr proto kernel_ra
       valid_lft 86268sec preferred_lft 14268sec
```

L'adresse MAC de `enp0s3` commence par `08:00:27`, le préfixe de VirtualBox : c'est bien l'interface de notre machine virtuelle. Pas besoin de sudo.

Utilisateurs humains

`cat /etc/passwd` liste tous les comptes de la machine. `cat /etc/login.defs` donne ensuite `UID_MIN` et `UID_MAX`, qui valent 1000 et 60000 : un utilisateur humain a forcément un UID compris entre ces deux valeurs. En cherchant dans `/etc/passwd`, nous voyons que seul `noa` est un utilisateur humain :

```
noa:x:1000:1000:noa,,,:/home/noa:/bin/bash
```

Les commandes `who` ou `users` permettent de le vérifier :

```
noa@debian-noa:~$ users
noa noa
```

Pas besoin de sudo.

Espace disque par partition

Pour obtenir l'espace disque par partition, nous avons utilisé `df` avec l'option `-h`, pour avoir des tailles lisibles :

```
noa@debian-noa:~/PreSaeNoa$ df -t ext4 -h
Sys. de fichiers  Taille  Utilisé  Dispo  Uti%  Monté sur
/dev/sda1           7,4G     1,8G   5,3G   25%  /
/dev/sda6            12G     1,4G   9,2G   14%  /home
```

Avec `-t ext4`, seules les vraies partitions de disque apparaissent. Pas besoin de sudo.

Les N processus les plus consommateurs de CPU

Nous avons utilisé `ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -n N` :

```
noa@debian-noa:~/PreSaeNoa$ ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -n
5
  PID %CPU %MEM COMMAND
 1324  1.2 22.3 MainThread
 1114  1.0  1.1 code-6d62b1d5da
 1129  0.7  6.1 MainThread
 1295  0.3  1.1 code-6d62b1d5da
```

L'option `--no-headers` enlève la ligne des noms de colonnes. Pas besoin de sudo.

Connexions réseau et processus associés

En parcourant `man ss`, nous avons trouvé la commande `ss -tuapn` :

```
noa@debian-noa:~/PreSaeNoa$ sudo ss -tuapn
Netid State  Recv-Q Send-Q Local Address:Port Peer Address:Port  Process
tcp   LISTEN 0      128    0.0.0.0:22         0.0.0.0:*          users:
(("sshd",pid=709,fd=6))
tcp   ESTAB  0      0      10.0.2.15:22       192.168.1.13:51770 users:
(("sshd-session",pid=1276,fd=7),("sshd-session",pid=1266,fd=7))
```

Le sujet demande que le script Ruby tourne sans root. Quand le processus n'apparaît pas, il faudra donc l'indiquer comme « inaccessible » sans planter. C'est pour voir toutes les informations que nous avons lancé l'exemple ci-dessus avec sudo.

Présence et statut des services clés (ssh, cron, docker)

Pour ces trois services, nous avons utilisé `systemctl is-active` et `systemctl is-enabled`. Comme le suggère le sujet, nous avons aussi testé `ssh.socket` :

```
noa@debian-noa:~/PreSaeNoa$ systemctl is-active ssh.socket
inactive
noa@debian-noa:~/PreSaeNoa$ systemctl is-enabled ssh.socket
disabled
```

Pas besoin de sudo.

2 Automatisation avec un script Ruby

Le script s'appelle `audit.rb`. Il rassemble toutes les informations de la partie 1, sans aucune interaction, et n'utilise que la bibliothèque standard de Ruby : `optparse` pour les options, `json` pour l'export et `open3` pour lancer les commandes. Il n'y a aucune gem à installer, ce qui simplifie aussi l'image Docker de la partie 3.

Dépôt Git : https://github.com/camil-1/projet_tuteure_craincourt_taleb (public).

Utilisation

```
ruby audit.rb -f texte                          # affichage lisible
ruby audit.rb                                   # JSON sur la sortie standard
ruby audit.rb -n 10 -s ssh,cron,docker,nginx    # 10 processus, services
choisis
ruby audit.rb -o audit.json                     # JSON enregistré dans un
fichier
ruby audit.rb -r /host -f texte                 # fichiers système lus
sous /host
```

Option	Rôle	Défaut
`-n`, `--nombre N`	Nombre de processus affichés (CPU et mémoire)	5
`-f`, `--format`	`json` ou `texte`	`json`
`-s`, `--services`	Liste de services séparés par des virgules	`ssh,cron,docker`
`-r`, `--racine`	Répertoire sous lequel lire `/etc`, `/proc`…	`/`
`-o`, `--sortie`	Enregistre le JSON dans ce fichier (prioritaire sur `-f` : rien n'est affiché à l'écran)	aucun

Limites et contrôle des paramètres

L'option `-n` est typée `Integer` avec `optparse` : le script refuse donc une chaîne de caractères. Les autres entrées, en revanche, ne sont pas vérifiées. Une valeur négative pour `-n` n'est pas contrôlée et provoque une exception non rattrapée (`ArgumentError` dans `take`). Un format inconnu pour `-f` bascule sans prévenir sur le JSON.

Les noms de services passés avec `-s` sont insérés tels quels dans la commande `systemctl`, que `Open3.capture3` lance sous forme d'une seule chaîne. Si un nom contient des caractères spéciaux du shell (un `;` par exemple), la commande est interprétée par un shell. Ce n'est pas seulement une erreur possible, c'est une injection de commande. Pour corriger, il suffirait de passer les arguments séparément (`capture3("systemctl", "is-active", unite)`) ou de valider les noms avec une expression régulière.

Enfin, `-o` est prioritaire sur `-f` : si on donne les deux, le JSON est écrit dans le fichier et rien ne s'affiche.

Organisation du code

Chaque information a sa propre méthode : `audit_machine`, `audit_uptime`, `audit_memoire`, `audit_reseau`, `audit_utilisateurs`, `audit_disque`, `audit_processus`, `audit_connexions` et `audit_services`. Chacune renvoie un Hash, ou un tableau de Hash pour les listes. Le résultat global est un Hash qui contient la date de l'audit (`Time.now.strftime('%Y-%m-%d %H:%M:%S')`), les paramètres utilisés et le retour de chaque méthode. Le JSON a donc ces clés de premier niveau : `date_audit`, `parametres`, `machine`, `uptime_et_charge`, `memoire`, `reseau`,

`utilisateurs`, `disques`, `processus_gourmands`, `connexions` et `services_cles`.

Ce qui change par rapport à la partie 1

Dans le script, certains choix diffèrent de la partie 1, pour deux raisons. L'option `--racine` oblige à lire des fichiers plutôt qu'à lancer des commandes (quand c'est possible), et un script a besoin d'une sortie facile à analyser.

Information	**Partie 1**	**Script**	**Pourquoi**
Nom de la machine	`hostname`	`/etc/hostname` sous la racine	`hostname` interroge le conteneur, pas l'hôte. Si le fichier manque, le script n'appelle pas la commande : le champ contient « Erreur: Fichier introuvable ».
Distribution	`cat /etc/os-release`	Même fichier, lu avec `PRETTY_NAME`	Pas de dépendance à `lsb-release`.
Noyau	`uname -r`	`uname -r`	Sortie propre, et c'est la même valeur dans un conteneur (noyau partagé).
Uptime et charge	`uptime`	`/proc/uptime` et `/proc/loadavg`	`uptime` lit ces fichiers et les met en forme. Les lire directement évite d'analyser du texte formaté (le séparateur décimal change avec la langue).
Mémoire et swap	`free -h`	`/proc/meminfo`	Même source que `free`. Utilisée = MemTotal − MemAvailable, comme `free`.
Réseau	`ip addr show`	`ip addr show`	Le script isole les adresses MAC, IPv4 et IPv6 de la sortie brute avec des expressions régulières. L'IPv6 est dans le JSON, mais le mode texte n'affiche que l'IPv4.
Utilisateurs	`/etc/passwd`, `who`, `users`	`/etc/passwd` filtré par `UID_MIN` et `UID_MAX`, puis `users`	Le script utilise directement la commande `users`.

Information	Partie 1	Script	Pourquoi
Disques	`df -h`	`df -P`	Le script lance `df -P` et garde seulement les lignes qui commencent par `^/dev/`.
Processus	`ps -eo pid,%cpu,%mem,comm --sort=-%cpu`	Même commande avec `--no-headers`	Le script garde les N premières lignes.
Connexions	`ss -tuapn`	`ss -tuapn`	Le script ignore lui-même la première ligne (l'en-tête).
Services	`systemctl is-active` et `is-enabled`	Idem, plus `ssh.socket` quand on teste `ssh`	Sur un Ubuntu récent, SSH peut être démarré par le socket.

Limites de `--racine`. Dans la version actuelle du script, `--racine` ne change le chemin de lecture que pour la machine (`/etc/hostname`, `/etc/os-release`), la mémoire (`/proc/meminfo`), l'uptime et la charge (`/proc/uptime`, `/proc/loadavg`) et les utilisateurs humains (`/etc/passwd`, `/etc/login.defs`). Les commandes pour le réseau (`ip`), les disques (`df`), les processus (`ps`), les connexions (`ss`), les sessions ouvertes (`users`) et les services (`systemctl`) ignorent ce paramètre et s'exécutent dans leur contexte normal. Cette limite a un impact direct sur les résultats de la partie Docker.

Disques et filtrage. Le script lance `df -P` et filtre les lignes avec la regex `^/dev/`. Ce filtre fonctionne bien sur une Debian, mais sur Ubuntu il liste aussi toutes les partitions virtuelles `/dev/loop*` créées par les paquets Snap, ce qui pollue la sortie. C'est une limite que nous assumons.

Connexions. Sans root, `ss -p` ne montre que les processus de l'utilisateur courant. Le script distingue trois cas. Si le processus est connu, il affiche son nom et son PID. Si le script ne tourne pas en root (`Process.uid != 0`) et que le processus manque, il écrit « inaccessible sans root ». S'il tourne en root et que le processus manque, c'est qu'aucun processus n'est associé au socket, par exemple pour une connexion en `TIME-WAIT`.

Robustesse

Le sujet impose que le script tourne sans root et ne s'arrête jamais sur une erreur. `executer_commande` lance les commandes avec `Open3.capture3` et renvoie la sortie standard, la sortie d'erreur et le code de retour. Une commande absente ou en échec produit un message d'erreur dans le résultat, et non une exception qui arrête tout. Elle ne protège pas, en revanche, contre l'injection de commande décrite plus haut. Chaque méthode vérifie avec `File.exist?` que ses fichiers sont présents et gère le cas où l'un d'eux manque. Les informations illisibles sans root, comme les PID via `ss`, sont signalées comme telles (« inaccessible sans root »).

Cette robustesse a ses limites : une valeur négative pour `-n` fait planter le script. Et si `ps` échoue, le mode texte affiche une ligne vide (`PID | CPU: %`) au lieu du message d'erreur, alors que le

JSON contient bien l'erreur.

3 Déploiement dans Docker

Le Dockerfile

L'image part de `debian:stable-slim` et installe uniquement ce que le script utilise : `ruby`, `iproute2`, `procps` et `systemd`. L'installation se fait avec `--no-install-recommends`, et `/var/lib/apt/lists` est supprimé dans la même couche pour garder une image légère. Le point d'entrée est `ENTRYPOINT ["ruby", "/app/audit.rb"]`, sous la forme exec : tout ce qui suit le nom de l'image dans `docker run` est transmis au script.

Options retenues pour auditer l'hôte

Le fichier `compose.yaml` regroupe toutes les options nécessaires pour auditer l'hôte malgré l'isolation du conteneur.

Option	Information concernée	Justification
`pid: host`	Processus, et `/proc/<pid>` pour `ss -p`	Le conteneur partage l'espace de noms PID de l'hôte.
`network_mode: host`	Interfaces et connexions	`ip` et `ss` voient la vraie pile réseau.
`/:/host:ro` avec `--racine /host`	`/etc`, `/proc`, montages	Tout l'hôte est visible, en lecture seule.
`/run/systemd:/run/systemd:ro`	Services	Contient le socket de systemd.
`SYSTEMD_IGNORE_CHROOT=1`	Services	Avec `pid: host`, `systemctl` se croit dans un chroot : il faut forcer son comportement.
`cap_add: SYS_PTRACE`	Processus des connexions	Lire `/proc/<pid>/fd` d'un autre utilisateur demande cette capacité.
`apparmor=unconfined`	Processus des connexions	Le profil `docker-default` interdit le ptrace.
`no-new-privileges:true`	Sécurité	Empêche toute élévation de privilèges.
`read_only: true` et `tmpfs: /tmp`	Sécurité	Verrouille le système de fichiers de l'image.
`./out:/out`	Export JSON	Seul dossier accessible en écriture, pour enregistrer le fichier.

Comparaison entre l'hôte et le conteneur

Nous avons lancé le script dans quatre situations : sur l'hôte sans root, sur l'hôte avec sudo (pour valider ce que voit le conteneur), dans le conteneur avec `docker compose`, et dans le conteneur seul sans aucune option, pour observer le comportement normal de l'isolation Docker.

Information	**Hôte (sans root)**	**Hôte (avec sudo)**	**Conteneur (avec options)**	**Conteneur (sans options)**
Nom de la machine	camil	camil	identique	28b11c92eb71
Distribution	Debian 13 (trixie)	Debian 13 (trixie)	identique	Debian 13 (trixie)
Noyau	6.12	6.12	identique	6.12
Date	2026-10-23 16:44:26	2026-10-23 16:45:00	2026-10-23 14:45:10	2026-11-03 18:44:04
Mémoire	RAM de l'hôte	RAM de l'hôte	identique	identique
Utilisateurs	Connecté	Connecté	Connecté	Aucun compte (objet vide `{}`)
Disques	`/` et `/boot/efi`	`/` et `/boot/efi`	Préfixe `/host` ajouté	Seulement le montage alloué par Docker (`/etc/hosts`)
Connexions	10 « inaccessible sans root »	Toutes visibles (avahi, cupsd…)	Toutes visibles	Aucune (`[]`)
Processus gourmands	ruby 85 %, etc.	ruby 85 %, etc.	docker-compose, ruby…	Uniquement ruby (PID 1) et ps (PID 24)
Services	cron et docker actifs et activés, ssh non installé	identique	identique	Erreur « System has not been booted with systemd as init system »

Ce que nous en retenons, ligne par ligne :

- **Nom de la machine.** Avec les options, `--racine` redirige bien la lecture vers `/host/etc/hostname`. Sans option, Docker génère son propre nom de machine à partir de l'identifiant du conteneur.
- **Distribution.** L'image de base (`debian:stable-slim`) est la même distribution que l'hôte, d'où le résultat identique. Si l'hôte était sous Ubuntu, la différence serait visible sans option.
- **Noyau.** Le noyau est toujours partagé entre l'hôte et les conteneurs, donc `uname -r` donne le même résultat, même sans option.
- **Date.** L'hôte est en UTC+2 et le conteneur, avec ou sans options, en UTC. Le format de `strftime` n'affiche pas le fuseau, mais l'heure diffère.
- **Mémoire.** Avec les options, `--racine` fait lire `/host/proc/meminfo`. Sans option, Docker ne virtualise pas `/proc/meminfo` par défaut : le conteneur affiche donc la RAM

totale de l'hôte.

- **Utilisateurs.** Avec les options, `-r /host` permet de trouver le compte dans `/host/etc/passwd`. La commande `users`, exécutée normalement, détecte aussi la session de l'hôte, grâce à l'espace PID partagé ou aux parties de l'hôte montées. Sans option, le compte n'est même pas trouvé, puisque le script lit le `/etc/passwd` du conteneur.
- **Disques.** C'est une limite du script : `df -P` ignore le paramètre `--racine`. Avec les options, le conteneur liste tous les disques montés, sous le préfixe `/host` ajouté par notre `compose.yaml`. Sans option, le conteneur n'a pas de disque à lui, seulement la couche overlay et les fichiers que Docker y monte.
- **Connexions.** Sans option, le conteneur est isolé au niveau réseau. Avec `network_mode: host` et `SYS_PTRACE`, le conteneur, qui tourne en root, remonte les mêmes connexions et les mêmes PID que l'hôte avec sudo.
- **Processus gourmands.** `ps` ignore la racine, mais avec `pid: host` il lit bien les processus de l'hôte. Sans option, on voit l'isolation du namespace PID : le conteneur ne voit que son propre processus et la commande lancée.
- **Services.** Sans le socket de systemd monté et sans `SYSTEMD_IGNORE_CHROOT`, `systemctl` échoue lamentablement dans le conteneur, car le PID 1 n'est pas systemd mais ruby.

Implications en matière de sécurité

Ces options rendent le conteneur beaucoup moins isolé que d'habitude. Un attaquant qui en prendrait le contrôle pourrait en profiter de plusieurs façons.

- `/:/host:ro` expose tous les fichiers de l'hôte en lecture. Le root du conteneur peut lire `/etc/shadow` et les clés SSH privées.
- `network_mode: host` supprime l'isolation réseau. Le conteneur peut écouter sur les ports de l'hôte ou capturer le trafic du réseau local.
- `apparmor=unconfined` retire les garde-fous que Docker applique par défaut.
- `/run/systemd:ro` n'empêche pas d'envoyer des ordres à systemd. On peut se connecter à un socket Unix même s'il est sur un montage en lecture seule. Comme le root du conteneur a l'UID 0, systemd lui obéit : il pourrait arrêter ou créer des services sur l'hôte, ce qui revient à en prendre le contrôle.

`no-new-privileges` et `read_only` limitent les dégâts, mais ne compensent pas ces accès. Pour un audit ponctuel sur sa propre machine, le risque est acceptable. Pour une automatisation, il vaut mieux lancer le script directement sur l'hôte avec cron.